Algorithm Comparison Report

Triangle Inequality Problem: Brute Force vs. Optimized Sorting

1. Overview

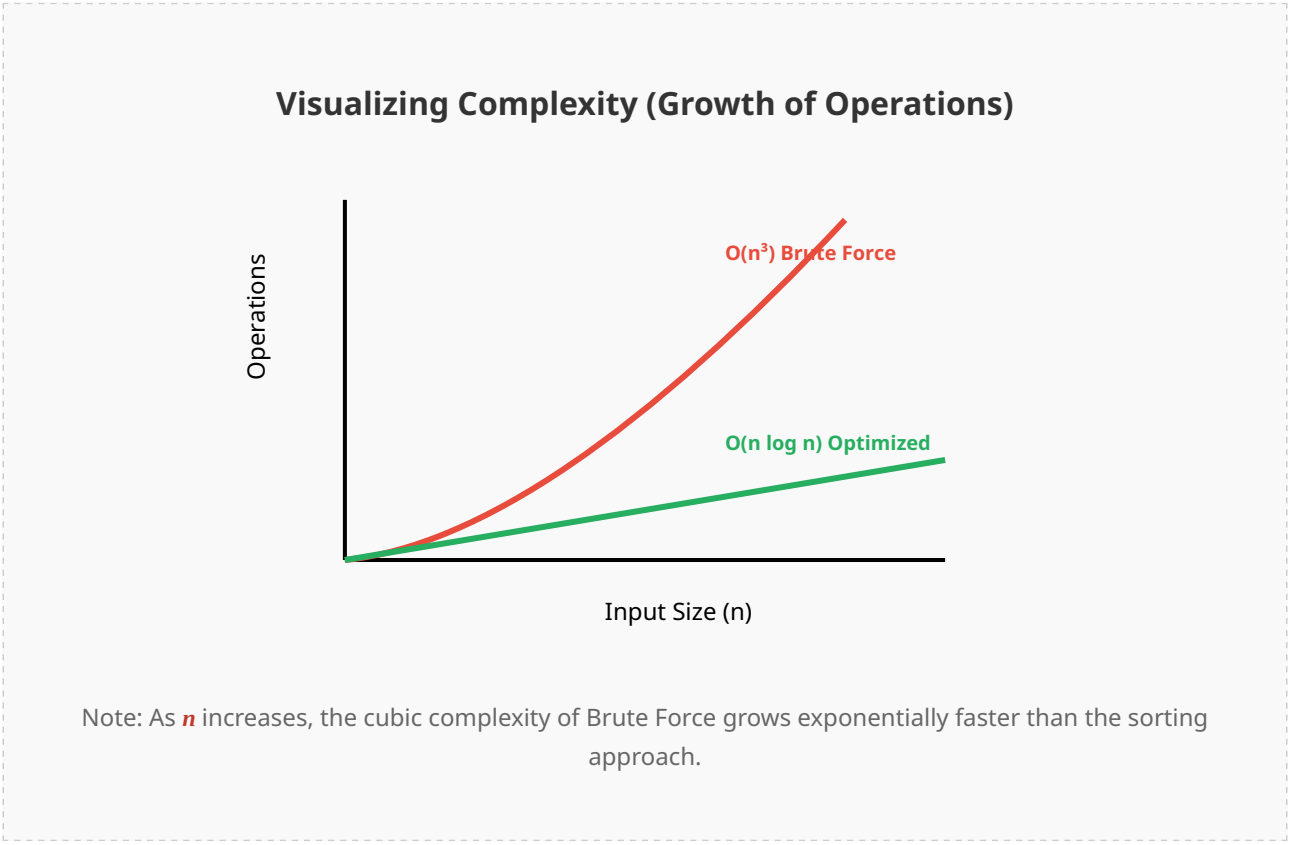
The goal of both algorithms is to determine if any three elements in a given array can form a valid triangle. According to the **Triangle Inequality Theorem**, three sides a, b, and c form a triangle if and only if:

$$a + b > c \quad \text{AND} \quad a + c > b \quad \text{AND} \quad b + c > a$$

2. Comparison Table

Feature	Brute Force Approach	Optimized Approach (Sorting)
Strategy	Checks every possible triplet (i, j, k).	Sorts the array first, then checks adjacent triplets.
Time Complexity	$O(n^3)$	$O(n \log n)$ (due to Merge Sort)
Space Complexity	$O(1)$ (In-place)	$O(n)$ (Temporary arrays for Merge Sort)
Implementation	Iterative (Triple nested loops).	Recursive (Merge Sort + Recursive scan).
Efficiency	Very slow for large n.	Highly efficient for large datasets.

3. Mathematical Complexity Growth



Note: As n increases, the cubic complexity of Brute Force grows exponentially faster than the sorting approach.

4. Key Differences Explained

Why Sorting Works

In a sorted array where $A[i] \le A[i+1] \le A[i+2]$, the conditions $A[i] + A[i+2] > A[i+1]$ and $A[i+1] + A[i+2] > A[i]$ are **always true**. Therefore, we only need to check the single condition: $A[i] + A[i+1] > A[i+2]$. This reduces the search space from all triplets to just adjacent triplets.

Recursive vs. Iterative

The second code uses **Merge Sort** and a **Recursive scan**. While recursion is elegant, the Brute Force iterative approach is simpler for very small arrays but fails to scale. The second code also utilizes dynamic memory allocation (`malloc`), making it more flexible for large datasets but requiring manual memory management (`free`).